Improving AprilTag Based Localization Performance and Accuracy for Robotics

Anthony R. Andrews

Palo Alto Middle College

Advanced Authentic Research,

Karen Logue

March 8, 2026

Abstract

This paper evaluates how AprilTag visual localization can be improved when considering inertial data and different Perspective-n-Point (PnP) algorithms in addition to AprilTag detector optimizations to reduce positional ambiguity and computational bottlenecks of the entire pipeline. By ultimately parallelizing the U-Michigan AprilTag detector and fusing specific OpenCV SolvePnP methods with inertial data, an improved localization pipeline is achieved especially in real-time edge environments. These improvements are open-sourced and ultimately support the democratization and adoption of AprilTag based systems in real-world applications.

Toward a Solution: Visual–Inertial Fusion and Multi-Camera PnP with Detector Parallelization

How do humanoid robots navigate crowded environments or drones land with centimeter-level precision? The answer lies in localization, the ability of a robot or computer system to estimate its position relative to the world around it using sensor data. In robotics and computer vision, one increasingly common method of localization uses AprilTags, high-contrast visual markers similar to QR codes that can be detected by a camera and used to calculate position and orientation in three-dimensional space. Since their introduction in 2011, AprilTags have become widely adopted in robotics research, educational competitions such as FIRST Robotics, autonomous drones, and commercial robotics platforms due to their relative simplicity, open-source accessibility, and strong positional accuracy (Olson, 2011; WPILib, 2025; Boston Dynamics, 2025).

Figure 1

Three detected AprilTags with decoded identifiers and a 3-dimensional bounding box representing a 3D estimate calculated by the PnP step.

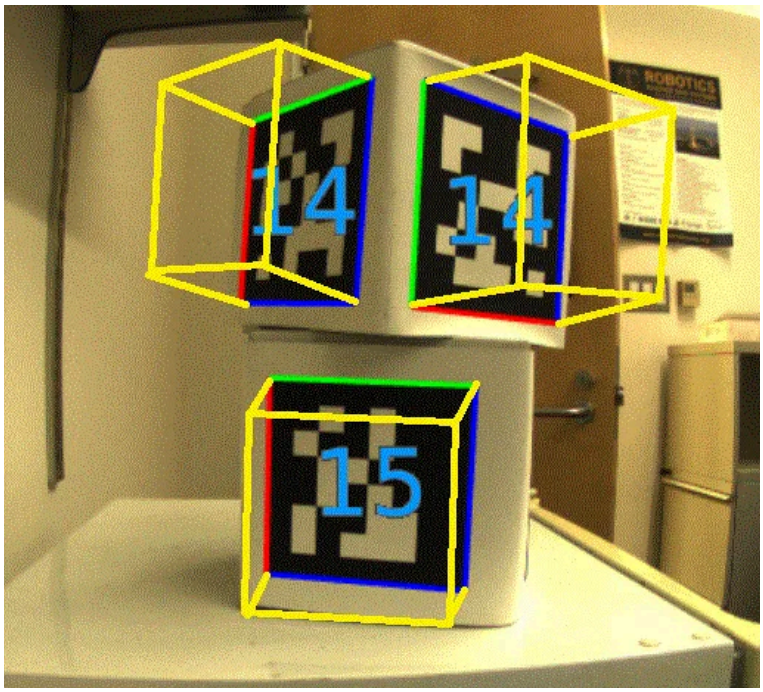
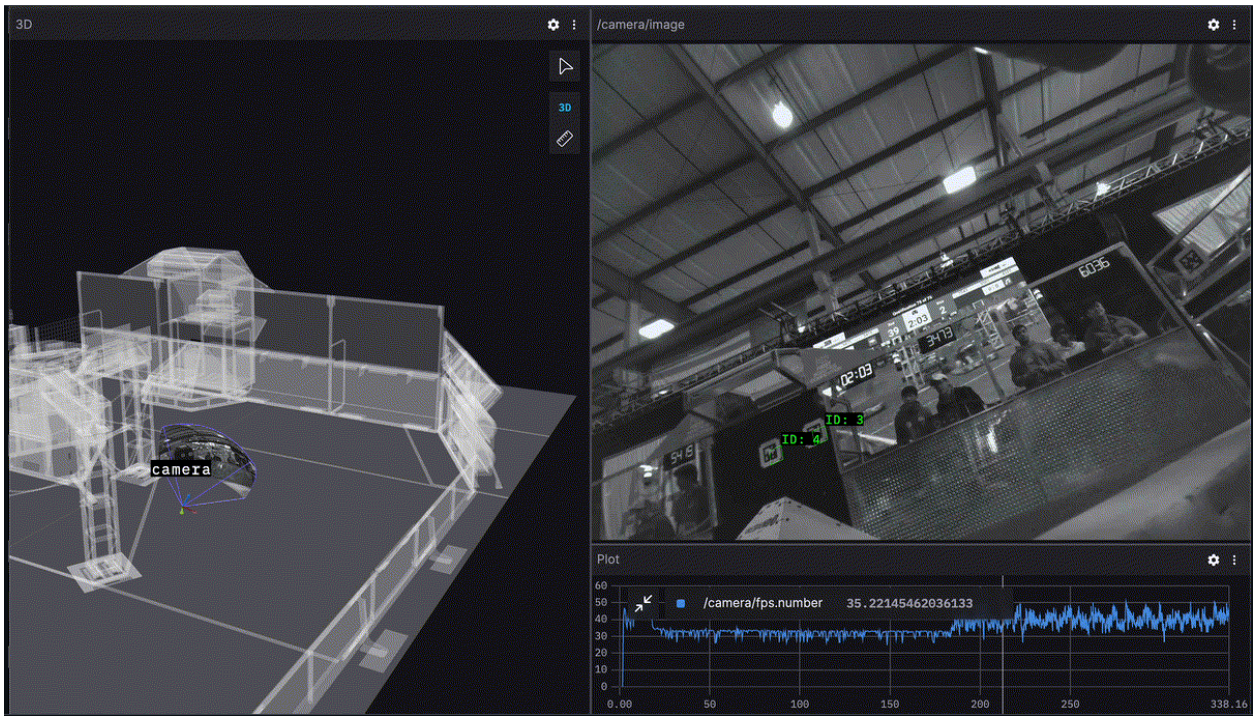


Figure 2

A 3d pose-estimate (3d position) calculated off real-world camera feed. Used in the FIRST robotics competition.



Note: The returned position does not include any information about surrounding objects and returns a relative position to detected AprilTag. For known environments, an additional map of surroundings is necessary such as in the preview on the left.

AprilTag localization works by detecting the corners of a tag within a camera image (Figure 1). and using mathematical pose-estimation techniques to determine the position and orientation of the camera relative to the tag (Figure 2). This process is commonly performed using Perspective-n-Point (PnP) algorithms implemented through computer vision libraries such as OpenCV. In real-time robotics systems, localization speed and reliability are both critical. A robot must not only determine where it is, but do so quickly enough to react to changing

conditions. Metrics such as latency (the time required to process a frame) and throughput or frames per second (FPS) are therefore important indicators of localization performance. Lower computational overhead generally allows a system to process more frames in less time, improving responsiveness and enabling deployment on lower-power embedded hardware commonly used in robotics systems.

Despite their widespread use, AprilTag systems still face two major limitations: positional ambiguity and computational performance scaling. Pose ambiguity occurs when multiple mathematically valid three-dimensional poses can be produced from the same two-dimensional camera image (Figure 3), particularly when viewing a planar tag at shallow angles. An analogy to these two problems would be in the case of identifying Waldo in a Where's Waldo image. The speed at which you can recognize Waldo per image is analogous to FPS. Likewise, attempting to locate Waldo in a cropped section of a "Where's Waldo?" The image without the surrounding context of the full scene creates multiple possible interpretations that may appear correct even though only one reflects reality. An AprilTag viewed from certain perspectives may produce two plausible pose estimates that both appear mathematically valid. In robotics applications, this ambiguity can lead to unstable localization, incorrect motion estimation, or "jittering" pose outputs during movement (Wu, Tsai, & Chien, 2014).

Figure 3

Two identical squares (simplification of the AprilTag pattern) with ambiguous 3D estimates.

Note: In both cases the calculated 3D estimate “looks correct”, but only will be accurate to the real-world.

At the same time, the AprilTag detector itself often represents the most computationally expensive stage of the localization pipeline. Detecting tags requires processing large quantities of image data in real time, often hundreds of millions of pixels per second. Although modern AprilTag detectors support multithreading, existing implementations frequently fail to scale across available CPU resources, limiting overall throughput and increasing latency. These constraints become especially important on embedded edge-computing platforms, where processing power, thermal limits, and power consumption are restricted.

Other localization systems, such as Simultaneous Localization and Mapping (SLAM), commonly improve reliability by combining visual data with inertial measurements from Inertial Measurement Units (IMUs) and data from multiple cameras. IMUs provide information about acceleration and orientation, allowing systems to use external motion data to stabilize visual pose estimates and have become ubiquitous in smartphones, and robotic platforms alike. Similar visual–inertial fusion approaches have been shown to improve robustness in broader robotics

localization systems (D'Alfonso et al., 2021; Scaramuzza, 2018), but comparable techniques remain relatively unexplored in open-source AprilTag pipelines. Likewise, while community discussions and recent software revisions suggest opportunities for improving AprilTag detector performance through process-level parallelization and code optimization, there has been limited empirical evaluation of these approaches in practical robotics environments.

This study investigated two primary research questions. First, can modifications to the Perspective-n-Point (PnP) stage reduce positional ambiguity in AprilTag localization? Second, can modifications to the AprilTag detection pipeline improve computational performance scaling and increase overall throughput? To address these questions, this research evaluated multiple PnP strategies, IMU-constrained pose filtering methods, detector parallelization techniques, and recent detector optimizations across desktop and embedded computing platforms. By exploring practical improvements using open-source tools and existing robotics hardware, this study aimed to determine whether meaningful gains in localization stability and real-time performance could be achieved without requiring entirely new algorithms or specialized hardware.

Technical Background and Existing Research

Definitions

AprilTags belong to the broader field of computer vision, a branch of computer science focused on enabling computers and robotic systems to interpret visual information from cameras and other imaging devices. In a typical AprilTag localization pipeline, a camera captures a stream of images referred to as frames. Each frame is processed individually by an AprilTag

detector, which identifies the location and identity of visible tags by locating their distinct high-contrast corners and decoding their internal patterns. The detector outputs the pixel coordinates of each detected corner within the image. These coordinates are then combined with camera calibration data and the known physical dimensions of the tag in order to estimate a three-dimensional pose using a Perspective-n-Point (PnP) algorithm. Pose estimation refers to determining both the position and orientation of the camera relative to the detected tag. Figure 2 illustrates these main two stages of the overall pipeline.

In robotics and real-time systems, localization performance is often evaluated using latency and throughput. Latency refers to the amount of time required to process a single frame, typically measured in milliseconds, while throughput is commonly measured in frames processed per second (FPS). Lower latency and higher throughput allow robotic systems to react more quickly to changes in their environment. These performance considerations become especially important in applications such as autonomous drones, robotic navigation, and industrial automation, where delayed localization can negatively affect control stability and precision. (Pant, Y. V., Abbas, H., Mohta, K., Nghiem, T. X., Devietti, J., & Mangharam, R., 2016) Computational overhead therefore becomes a critical constraint, particularly on embedded systems with limited processing power, thermal capacity, or power availability.
Several factors impact the FPS of the AprilTag detector algorithm including but not limited to:

- Resolution - the number of pixels per frame
- Decimate - the internal downsampling of each frame to improve performance at the cost of accuracy
- Number of threads - how many processor threads are allocated to detect on each frame

- Processor speed

Existing Research

Compared to more complex localization approaches such as Simultaneous Localization and Mapping (SLAM), AprilTag systems are often favored for their simplicity, precision, and accessibility. SLAM systems estimate position while simultaneously constructing a map of the surrounding environment using sensor data from cameras, LiDAR, and inertial sensors. Although SLAM provides flexibility in unknown environments, it generally requires substantially greater computational resources and introduces additional sources of drift and uncertainty over time (MathWorks, n.d.). In contrast, AprilTags provide fixed reference points that allow for highly accurate pose estimation when the tags are visible within the camera frame. This balance between simplicity and accuracy has contributed significantly to the adoption of AprilTags across both research and industry applications.

Evidence from industry and educational robotics demonstrates the growing importance of AprilTag-based localization. In educational robotics, WPILib documentation describes AprilTags as a core component of modern robot localization systems used throughout the FIRST Robotics Competition, where robots use tags placed around the field to estimate their position and align with scoring objectives (WPILib, 2025). Outside of educational robotics, researchers at the University of California, Santa Barbara integrated visual fiducials into autonomous drone navigation systems to enable precision landings in environments where GPS was unavailable or unreliable (Dupree et al., 2019). Commercial robotics companies have also adopted similar localization techniques. Boston Dynamics documentation references the use of fiducial-assisted

localization within larger navigation workflows for autonomous robots operating in real-world environments (Boston Dynamics, 2025). More recently, software platforms such as Radiance Fields' Postshot integrated AprilTag support into photogrammetry and three-dimensional reconstruction workflows, citing improvements in tracking reliability and computational efficiency on consumer-grade hardware (Rubloff, 2025). Together, these examples demonstrate that AprilTags have become an important and widely adopted localization technology across both academic and commercial domains.

Despite this widespread adoption, several important technical limitations remain unresolved. One of the most significant is pose ambiguity. Because AprilTags are planar markers viewed through a two-dimensional camera image, certain viewing angles can produce multiple mathematically plausible three-dimensional pose solutions. This issue becomes especially problematic at shallow viewing angles where visual perspective information is reduced. Under these conditions, two different poses may appear equally valid despite only one accurately representing the real-world orientation of the tag. Collins and Bartoli's Infinitesimal Plane-Based Pose Estimation (IPPE) method demonstrated that planar PnP problems inherently contain this two-fold ambiguity and proposed returning both candidate solutions alongside associated reprojection errors rather than forcing a single potentially unstable solution (Collins & Bartoli, 2014). Wu, Tsai, and Chien (2014) similarly observed that pose ambiguity can lead to unstable pose tracking and "jittering" in real-time applications when insufficient contextual information is available to disambiguate solutions.

Another major limitation involves computational scalability within the AprilTag detector itself. Detecting AprilTags requires analyzing large quantities of image data in real time, often involving hundreds of millions of pixels per second. Although existing AprilTag

implementations support multithreading, later experimental benchmarks in this paper suggest that detector performance frequently plateaus beyond approximately four to six CPU threads, resulting in poor overall processor utilization. As a consequence, real-time throughput may remain lower than expected even on systems with many available CPU cores. These performance limitations become especially restrictive on embedded edge-computing platforms commonly used in robotics, where available compute resources are substantially lower than on desktop systems.

Existing research suggests several possible directions for addressing these limitations, but important gaps remain. Visual–inertial fusion methods used in SLAM systems have demonstrated that inertial measurements from IMUs can stabilize pose estimation by providing external orientation information alongside visual data (D'Alfonso et al., 2021; Scaramuzza, 2018). Some commercial AprilTag-based localization systems have also begun incorporating IMU heading information to reduce ambiguity during localization, although implementation details and supporting experimental data are often limited or proprietary (Limelight Vision, 2024). There remains limited empirical evaluation of these approaches within open-source AprilTag pipelines, particularly on embedded robotics hardware.

This gap in existing research motivated the present study. While prior work has established that AprilTags are highly effective for localization, less attention has been given to how ambiguity reduction and computational scaling strategies can be practically implemented within accessible open-source systems. This study therefore focused on evaluating whether IMU-constrained pose selection, detector parallelization, and detector optimization techniques could measurably improve localization reliability and computational performance in real-world robotics contexts.

Research Methodology

This study employed a quasi-experimental design to evaluate improvements to an AprilTag-based localization pipeline. Two independent variables were tested: (1) computational performance optimizations within the AprilTag detection pipeline and (2) pose estimation strategies within the Perspective-n-Point (PnP) stage. A baseline AprilTag detection and pose estimation system served as the control condition for all comparisons.

Experimental Setup

A baseline AprilTag detection and pose-estimation pipeline served as the control condition for all experiments. Rather than modifying the underlying mathematical implementations of the OpenCV PnP solvers or rewriting the AprilTag detector itself, this study focused on practical improvements that could realistically be integrated into existing open-source robotics software while maintaining compatibility and maintainability.

All experiments were conducted on two hardware platforms representing both desktop-class and embedded robotics systems. The first platform used an AMD Ryzen 5 5600H processor, while the second used a Rockchip RK3588 embedded platform commonly found in robotics and edge-computing applications. Both systems processed identical datasets and software configurations to ensure comparability across experiments.

The software stack consisted primarily of Python, OpenCV, and the University of Michigan AprilTag detector alongside custom experimental scripts used for automation and data collection. It is worth noting that other AprilTag detector codebases such as ArUco were considered but ultimately not used for its lower positional accuracy overall (Rufus, 2019;

PhotonVision, 2024). Python was selected due to its widespread use in computer vision research and strong ecosystem support for rapid experimentation (Raschka, Patterson, & Nolet, n.d.). All experimental code, datasets, and raw performance measurements were version-controlled through a public GitHub repository to support transparency and reproducibility.

To isolate software variables from environmental variables, two categories of test data were used. Simulated images generated within a voxel-based game engine provided idealized pin-hole camera model conditions free from real-world lens distortion or sensor noise that the PnP algorithms expect. These simulated environments also provided known ground-truth pose information useful when evaluating pose ambiguity behavior. Real-world testing used pre-captured uncompressed images of printed AprilTags under varying lighting conditions and viewing angles. These datasets introduced practical complications such as lens distortion, sensor noise, and uneven lighting that better represented real robotics environments.

The decision to use both simulated and real-world datasets was intended to balance experimental control with realism. Simulated images isolated the algorithms themselves, while real-world images verified that improvements remained effective outside idealized conditions.

Detector Performance Experiments

The first experimental branch focused on Research Question 1: improving computational performance within the AprilTag detection pipeline.

Early baseline experiments revealed that increasing the thread count of a single detector instance produced diminishing performance returns beyond approximately four to six threads. Although

no prior research explicitly documented this limitation, repeated testing consistently demonstrated that throughput plateaued despite additional CPU resources remaining available.

To determine whether this bottleneck was caused by image-processing workload or by the detector's internal threading behavior, several variables were isolated independently:

- detector thread count
- number of detector processes
- detector decimation level
- image resolution
- detector software version

All experiments kept image datasets and detector configurations constant while varying only the variable under investigation.

Before testing parallelization behavior, additional experiments evaluated the impact of image resolution and decimation on detector throughput. Decimation reduces image resolution prior to processing, decreasing the number of pixels analyzed by the detector. Results showed that lower decimation and higher resolutions increased total pixels processed per second, while higher decimation significantly increased FPS at the cost of reduced image detail.

These findings established that detector workload alone did not fully explain the observed scaling limitations.

The next phase evaluated thread scaling within a single detector instance compared to multi-process parallelization using multiple independent detector instances.

Throughout testing, the Linux monitoring tool htop was used to measure CPU utilization across all cores. CPU utilization data was compared directly against detector throughput to determine how effectively available compute resources were being used.

The underlying hypothesis was that several major stages of the AprilTag pipeline, including decimation, adaptive thresholding, contour extraction, and tag decoding, are only parallelizable to a limited degree within a single process. Beyond this point, overhead and internal scheduling bottlenecks reduce the effectiveness of adding additional threads[1].

Finally, experiments compared the stable AprilTag detector release (3.4.5) against a newer public repository commit approximately seven months newer (3.4.5a). Although this study did not attempt to reverse-engineer or isolate the exact low-level optimizations responsible for the performance gains, the newer implementation consistently demonstrated measurable throughput improvements. These gains likely originated from community-driven improvements such as cache locality optimizations, memory-access improvements, and compiler-level efficiencies. (Badhai, A., 2025)

Importantly, this study deliberately chose not to modify the detector's internal implementation. The goal was not to redesign the algorithm itself, but rather to evaluate how practical systems-level improvements and recent community optimizations could improve real-world performance while remaining compatible with existing open-source ecosystems.

[1] Further research and reverse-engineering is needed to better understand the working of the U-Michigan Apriltag detector threading bottlenecks.

Pose Estimation (PnP) Experiments

The second experimental branch focused on Research Question 2: reducing pose ambiguity in planar AprilTag localization. When observing a flat AprilTag at shallow angles, the PnP step may produce multiple mathematically valid pose solutions due to the symmetry of planar geometry (Wu, Tsai, & Chien, 2014; Collins & Bartoli, 2014). This ambiguity can cause pose estimates to "flip" between orientations across frames, producing unstable localization behavior.

An analogy can be made to identifying Waldo in a crowded image. If only a small portion of Waldo is visible from an unusual angle, multiple possible interpretations of where Waldo is relative to the scene may appear correct until additional context is introduced. Similarly, the PnP solver may determine several poses that all appear mathematically plausible from the camera's perspective under limited information scenarios even though only one matches the real-world orientation (such as single tags at shallow angles).

Experiments focused primarily on two OpenCV PnP solvers: IPPE and SQPnP. IPPE was selected because its planar closed-form formulation explicitly returns two candidate pose solutions alongside their associated reprojection uncertainties (Collins & Bartoli, 2014). SOLVEPNP_ITERATIVE was selected as a comparison because it typically collapses ambiguous observations into a single dominant estimate output and is the default of software such as PhotonVision.

Test images included direct-on and shallow-angle observations of single AprilTags under both simulated and real-world conditions. Real-world tests used a printed AprilTag mounted on

flat polycarbonate at an approximate distance of four meters under lighting conditions ranging from ambient indoor sunlight to direct CFL illumination.

Accuracy was evaluated using:

- ambiguity frequency
- pose stability across frames
- qualitative observations of pose jitter
- separation between candidate pose headings

Across repeated shallow-angle observations, the IPPE solver consistently returned two clearly separated candidate poses. In many cases, the two solutions represented substantially different heading orientations despite both appearing visually correct from the perspective of the planar image.

The logic behind the proposed solution was that if an external sensor such as an IMU could reliably provide heading information, this external orientation reference could be used to reject the incorrect pose solution. Due to project time constraints, a fully integrated real-time IMU fusion pipeline was not implemented during this study. Instead, simulated heading constraints were applied as a proof-of-concept demonstration.

Simulated heading constraints demonstrated that external orientation data could reliably reject incorrect pose solutions. This result suggested that preserving ambiguity may ultimately be more useful than hiding ambiguity. By explicitly exposing both mathematically valid solutions, IPPE enabled additional sensor data to participate in pose selection rather than prematurely collapsing the ambiguity into a single estimate.

Data Collection and Metrics

For each experiment, the following metrics were recorded:

- frames per second (FPS)
- frame processing latency (milliseconds per frame)
- Total CPU utilization (as a percentage of each thread[1])
- ambiguity uncertainty
- qualitative stability observations

All experiments were repeated multiple times under identical conditions to reduce measurement noise caused by operating-system scheduling and background processes. Final reported values represent averages across repeated runs.

While FPS served as a primary performance metric, this study also evaluated pixels processed per second. FPS alone can be misleading because lower-resolution or highly decimated images require significantly less computational work per frame. Measuring total pixel throughput provided a more meaningful estimate of detector efficiency across varying resolutions and decimation levels.

Raw datasets, performance logs, experimental scripts, and additional figures are linked through the project GitHub repository referenced in the sources section.

[1] In the case of processors with more than one thread, the maximum total percentage will be greater than 100%.

Challenges Faced

Several challenges arose during implementation and evaluation of the detection and pose estimation pipelines.

Limited Documentation and Black-Box Behavior

Both the AprilTag detector and OpenCV PnP pipeline functioned largely as black-box systems. Limited documentation on internal optimizations made it difficult to understand failure cases or performance differences without extensive empirical testing.

Understanding PnP and Geometric Foundations

Interpreting pose output required a working understanding of Perspective-n-Point (PnP), camera projection, and reprojection error. Debugging often involved separating detection errors from solver instability, which was non-trivial due to tightly coupled pipeline outputs.

Measurement Consistency

Benchmarking required strict control of:

- input resolution and decimation settings
- detector parameters across implementations
- CPU load and thread allocation

Even small system-level variations (background processes, thermal throttling) introduced noise in FPS and latency measurements, requiring repeated trials and averaging.

Real-World Variability

Performance degraded inconsistently under real-world conditions such as motion blur, lighting variation, and partial tag occlusion.

Pipeline Integration and IMU Fusion

Integrating IMU data with vision-based pose estimates introduced synchronization and drift issues. Initial direct fusion attempts were unstable, requiring separation into a dedicated experimental testing pipeline.

Parallelization Limits

Multi-threading improved throughput, but gains were non-linear. After a threshold, inter-process overhead and CPU contention reduced returns, and additional threads sometimes degraded performance.

Timeline

- Months prior (2025 – early 2026): Literature review, system design, and background research on fiducial markers, PnP, and vision pipelines
- Early March 2026: Initial implementation and baseline detector testing

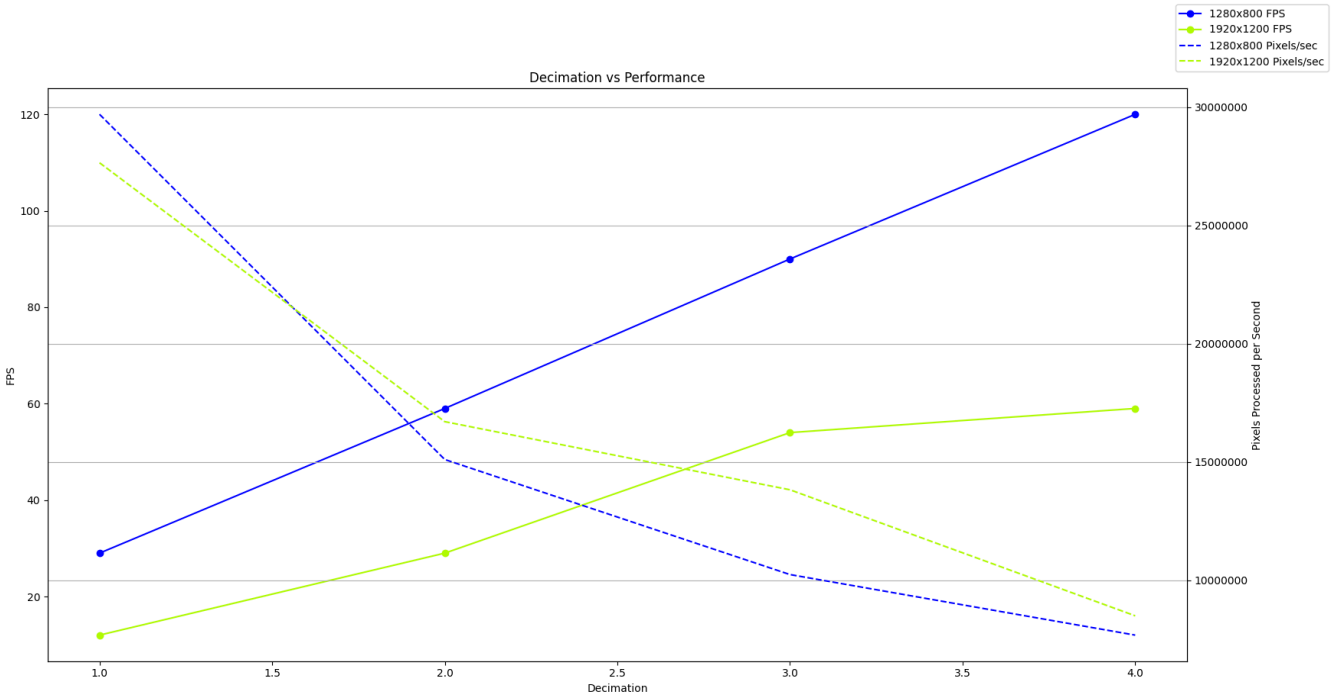
- March 4 – March 11, 2026: Core experimental benchmarking and data collection phase (ArUco vs AprilTag vs PnP pipeline evaluation)
- Post-March 11, 2026: Analysis, result refinement, and integration of findings into final report

Results and Findings

Decimation Performance Results

Figure 4

Effect of image decimation on FPS and pixel throughput across resolutions



Initial experiments evaluated how image resolution and decimation affected detector workload and throughput. Results showed that increasing decimation substantially increased FPS by

reducing the number of pixels processed per frame. However, lower decimation and higher resolutions consistently achieved greater total pixel throughput.

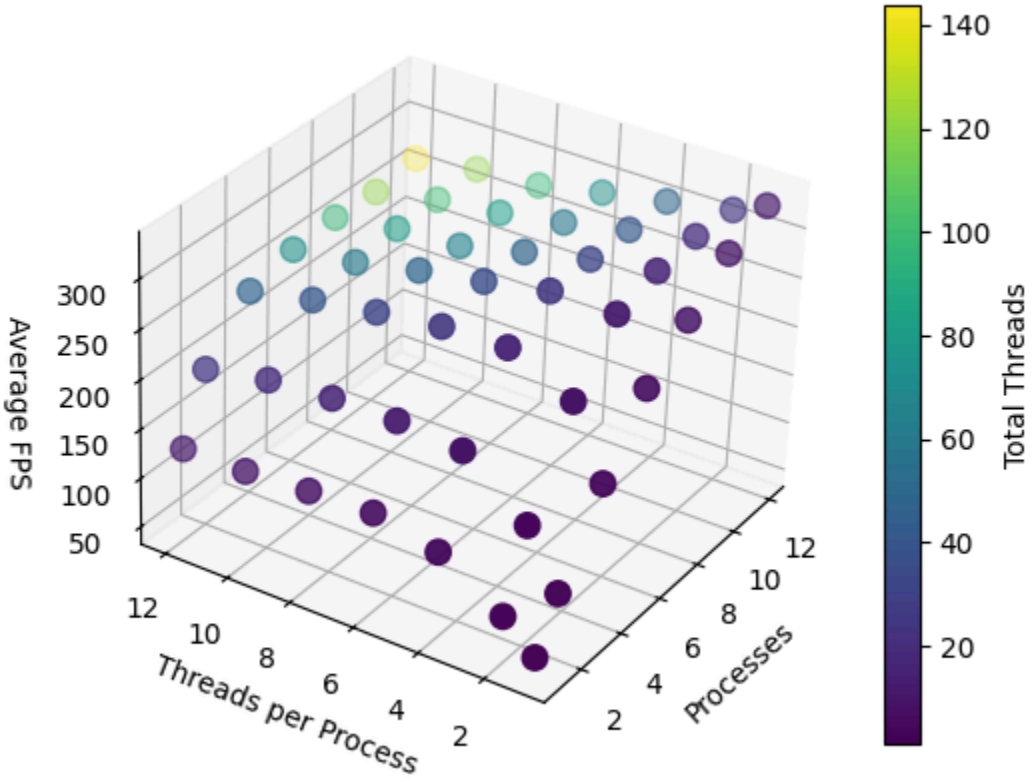
These findings demonstrated that FPS alone was not a sufficient metric for evaluating detector efficiency. While highly decimated images produced faster frame rates, they often processed substantially less visual information overall. Pixel throughput therefore provided a more accurate representation of actual detector workload.

The results also established that image-processing workload alone did not fully explain the scaling limitations observed in later experiments.

Detector Parallelization Results

Figure 5

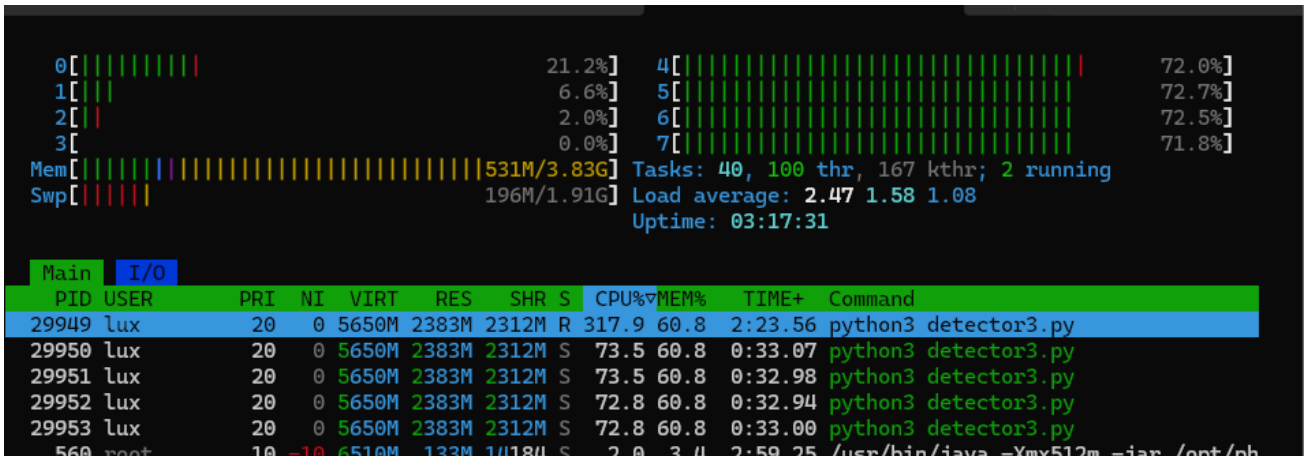
U-Mich *AprilTag detector performance scaling*



Note: Test run on 5600H 12-thread test system using test video1

Figure 6A

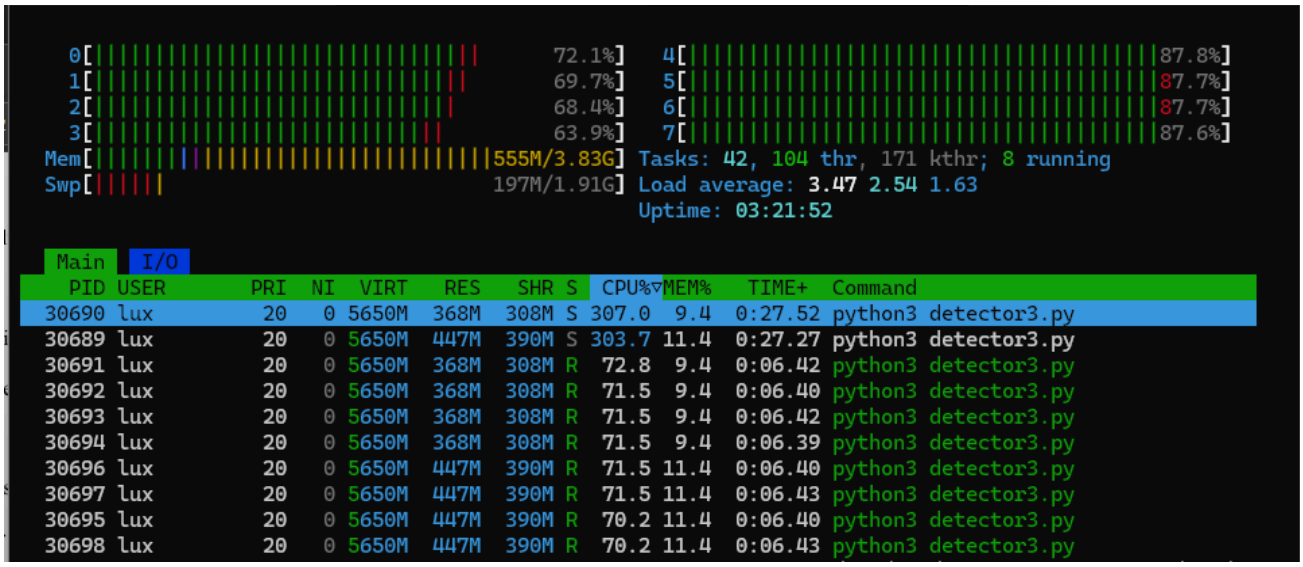
RK3588 test platform processor utilization running a single AprilTag detector instance with four threads.



Note: Shown is one instance of "python3 detector3.py" with 4 threads assigned. The CPU% column represents utilization of a single thread. Overall usage is roughly 40% of the total 800% compute available (8-threads). This result yields around 12fps at 1920x1200 Decimate 1 Test video1.

Figure 6B

RK3588 test platform processor utilization running two AprilTag detector instances with four threads each.



Note: Shown is two instances each with 4 threads assigned, the CPU% column indicates usage to remain around 80% total. This result yields around 20fps at 1920x1200 Decimate 1 Test video1.

Figure 6C

RK3588 test platform processor utilization running two AprilTag detector instances with four threads each.

Note. Shown is A single AprilTag detector instance failed to fully utilize available CPU resources, even when increasing internal thread counts. CPU utilization plateaued before reaching full system capacity, indicating inefficiencies in the detector's internal threading model.

Experimental results showed that a single AprilTag detector instance failed to fully utilize available CPU resources even when assigned additional threads. Performance improved initially as thread count increased, but quickly reached diminishing returns despite unused CPU capacity remaining available.

A useful analogy is a grocery-store checkout line. A single heavily-threaded detector behaved similarly to one large checkout lane with many workers attempting to operate within the same constrained workflow. Eventually, adding additional workers no longer improved throughput because all tasks still depended on the same shared queue and coordination overhead.

In contrast, running several smaller detector processes behaved more like opening multiple independent checkout lanes. Workloads became distributed across several independent processing pipelines, allowing overall CPU utilization to increase substantially.

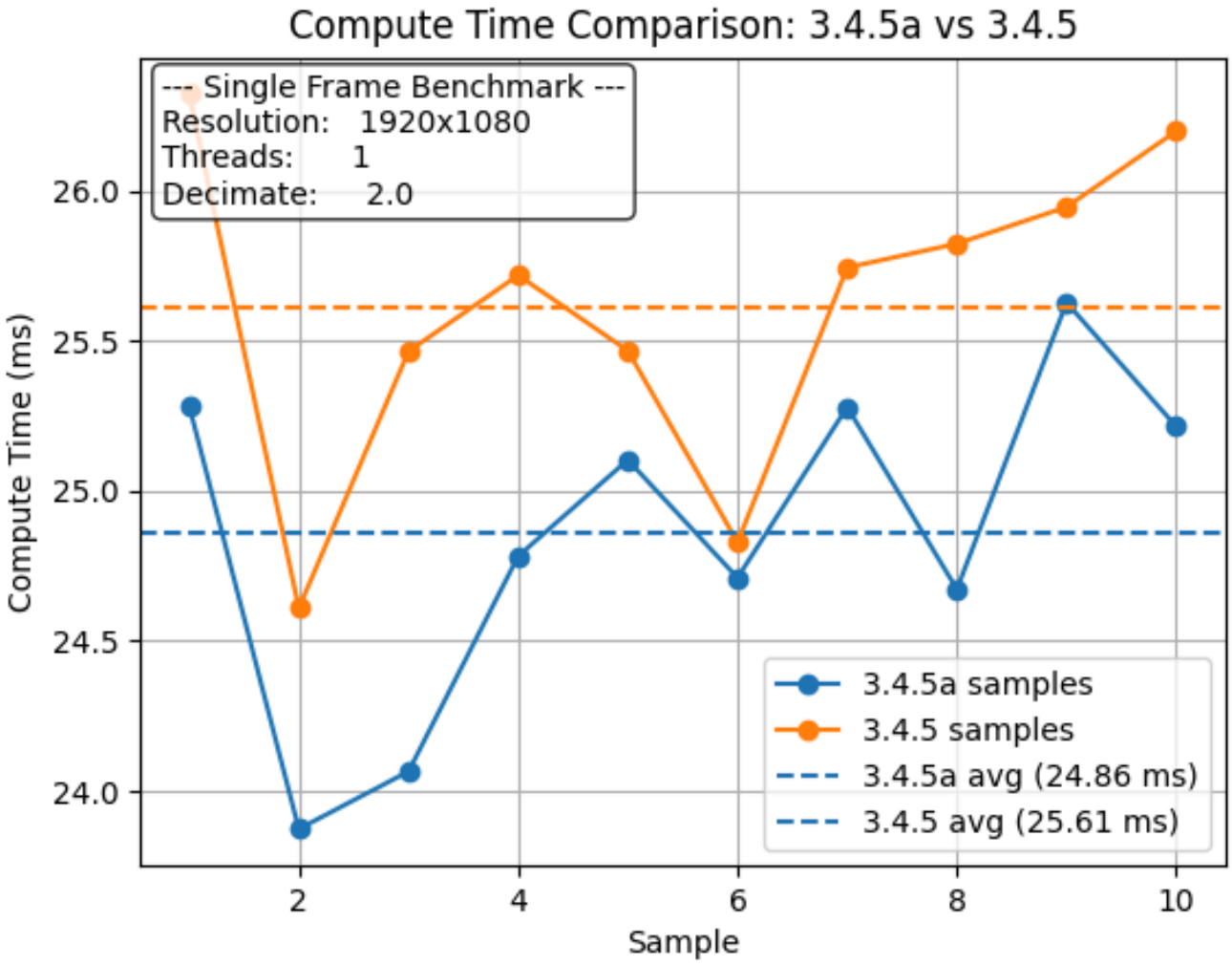
On the RK3588 platform, running two detector processes with four threads each nearly doubled both FPS and total CPU utilization compared to a single detector instance using the same total number of threads. CPU utilization increased from roughly partial utilization to near full system utilization.

These findings suggest that the primary bottleneck was not the detector algorithm itself, but rather how the detector internally distributed work across threads. Process-level parallelization proved significantly more effective than relying solely on the detector's built-in multithreading behavior.

Detector Version Optimization Results

Figure 7

Performance comparison between AprilTag release versions (3.4.5 vs updated 3.4.5a).



Note: Lower is better

Updating the AprilTag detector from the stable 3.4.5 release to a newer public repository commit approximately seven months newer produced measurable performance improvements across all test conditions.

Although the detector algorithm itself remained fundamentally unchanged, the updated implementation consistently reduced frame-processing latency and increased achievable FPS.

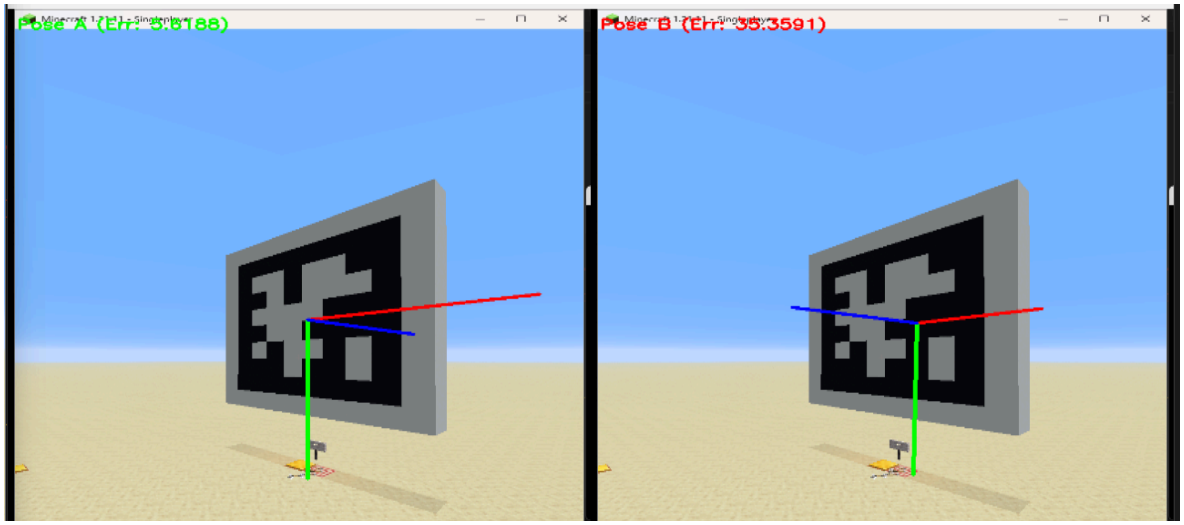
This suggests that small community-driven improvements to memory access patterns, cache locality, and low-level execution behavior can accumulate into meaningful real-time performance gains.

Because the focus of this study was practical systems-level optimization rather than reverse-engineering the detector implementation itself, the exact internal optimizations were not investigated in detail. However, the results highlight an important advantage of open-source software ecosystems: incremental community optimizations can significantly improve performance over time even without major algorithmic redesigns.

PnP Solver Ambiguity Results

Figure 8

Both returned pose of the IPPE with the 3D Axis indicator overlayed



Note: Utilizing the IPPE SolvePNP method

Additional experiments evaluated how different PnP solvers handled planar AprilTag observations under ambiguous viewing conditions.

The IPPE solver consistently returned two distinct simultaneous pose solutions when the tag was viewed at shallow angles. These solutions were typically separated by significantly different heading orientations while still appearing visually plausible from the camera's perspective. This behavior matched the theoretical expectation that planar pose estimation inherently produces a two-fold ambiguity (Collins & Bartoli, 2014).

In contrast, SOLVEPNP_ITERATIVE generally returns a single dominant pose estimate as a result of its iterative solver nature which mathematically only generates a single pose at a time.

An analogy can again be made to locating Waldo in a crowded image. One approach is to acknowledge that two possible Waldos match the limited visual evidence and then use additional context to decide which one is correct. Another approach is to immediately guess one answer and ignore the ambiguity entirely. The experiments suggested that the first strategy ultimately produced more stable results.

The key finding was that preserving ambiguity proved more useful than hiding ambiguity. Because IPPEs closed-form solver method explicitly returned both mathematically valid solutions, external orientation data could later be used to select the physically correct pose.

Although a fully integrated IMU fusion pipeline was outside the scope of this study due to time constraints, simulated heading constraints successfully rejected incorrect pose solutions across all observed ambiguous test cases. This demonstrated that external orientation information can effectively disambiguate planar pose estimates without fundamentally modifying the underlying PnP algorithm itself.

Conclusion and Analysis

This study investigated whether practical systems-level improvements could improve the performance and reliability of open-source AprilTag localization pipelines and was able to do so without fundamentally redesigning the underlying algorithms.

The findings demonstrated that the computational limitations of the AprilTag detector were not solely caused by algorithmic complexity, but also by inefficient utilization of available compute resources. A single heavily-threaded detector instance consistently reached diminishing returns despite unused CPU capacity remaining available. By restructuring the workload into multiple smaller detector processes, overall throughput and CPU utilization improved substantially, particularly on embedded robotics hardware such as the RK3588.

The study also demonstrated that planar pose ambiguity is not necessarily a flaw that must be mathematically eliminated. Instead, explicitly preserving multiple valid pose solutions enabled external orientation information to participate in the disambiguation process. IPPE's dual-solution closed-form output proved especially well-suited for this approach because it exposed ambiguity directly rather than disregarding it mathematically.

Together, these findings suggest that meaningful improvements to robotics localization systems can often come from practical systems engineering decisions rather than entirely new algorithms. Improvements in workload orchestration, solver selection, and sensor integration may provide significant real-world benefits while remaining compatible with existing open-source software ecosystems.

More broadly, the study reinforces the importance of open-source development within robotics and computer vision. The measurable gains observed from a newer community-maintained

detector revision demonstrate how incremental public contributions can meaningfully improve real-world performance over time.

Implications and Next Steps

The results of this study have implications for robotics systems requiring reliable real-time localization under limited computational budgets and/or systems with the increasingly common inclusion of built-in IMU sensors.

Broader Impact

Ultimately the community efforts and improvements of AprilTag based localization technology will enable automation in new and helpful fields. AprilTags have already been extensively adopted in fields such as autonomous drone navigation, augmented reality to help those with impairments, and scientific research. The bottleneck in innovation becomes the accumulation of small improvements such as those suggested in this paper and the open-source accessibility alongside helpful documentation and education all helps to solve what previously was a difficult barrier.

Potential Future Work

Several areas for future work remain.

Most importantly, future research should integrate a real-time physical IMU directly into the localization pipeline rather than relying on simulated heading constraints. While this study demonstrated the feasibility of IMU-constrained pose filtering as a proof-of-concept, a fully

integrated visual–inertial fusion system would allow ambiguity reduction to be evaluated under dynamic real-world motion.

Additional future work could include:

- dynamic testing on moving robotic platforms
- multi-camera PnP orchestration for improved spatial coverage
- GPU acceleration of detector workloads
- optimization of single process AprilTag detection
- evaluation of hardware-specific SIMD or NPU optimizations
- SLAM based systems as a more powerful alternative to AprilTags

Future studies could also investigate whether preserving ambiguity explicitly may improve robustness in other computer-vision systems beyond AprilTag localization.

Overall, this research demonstrates that practical improvements to AprilTag-based localization are achievable through careful systems engineering, workload orchestration, and sensor-assisted filtering while remaining compatible with existing open-source robotics ecosystems.

References

2025 Carnegie Mellon University. (n.d.). *What are AprilTags? | CS-STEM Network*. CS-STEM Network. Retrieved October 24, 2025, from https://www.cs2n.org/u/mp/badge_pages/3255

This introductory module explains the concept of AprilTags and their role in computer vision and robotics education. It provides accessible explanations of tag identification, pose estimation, and practical classroom uses. Useful as a general overview of AprilTag principles for beginners or educational purposes.

Andrews, A. (2026, May 13). AprilTag. *GitHub*. Retrieved May 13, 2026, from https://github.com/Anthony-Andrews/AprilTag

Repository for all the experimental code and results gathered throughout this paper.

Badhai, A. (2025, May 23). Cache locality. *Medium*. https://medium.com/@arun.badhai/cache-locality-6bd527d157e7

This article explains the concept of cache locality and how modern CPUs rely on spatial and temporal locality to improve performance. The author describes how processors access data in chunks called cache lines and why programs that access nearby or recently used memory locations run significantly faster. The source connects these hardware principles to practical programming decisions, such as array traversal order and data structure layout, showing how inefficient memory access patterns can create major slowdowns even when algorithms remain unchanged. This is one of many tricks used by maintainers of the AprilRobotics/apriltag codebase to make incremental optimizations.

Boston Dynamics. (2025). *GraphNav Initialization*. Boston Dynamics Developer Documentation. Retrieved October 12, 2025, from https://dev.bostondynamics.com/docs/concepts/autonomy/initialization.html

Describes the GraphNav system's initialization process for robot navigation, including localization and mapping concepts. Relevant as an applied example of SLAM and visual fiducial usage in commercial robotics, showing how localization systems integrate with perception data in real-world autonomous navigation.

Collins, T., & Bartoli, A. (2014). Innitesimal Plane-based Pose Estimation [PDF]. https://encov.ip.uca.fr/publications/pubfiles/2014_Collins_etal_IJCV_plane.pdf

This paper introduces IPPE as a closed-form, non-iterative solution for estimating the 6-DoF pose of a camera from coplanar point correspondences using the homography induced by the plane. A key result is that the formulation inherently produces a two-fold ambiguity in rotation, corresponding to a reflection symmetry of the planar configuration about a plane through the camera line-of-sight. The method exploits the geometric constraint of co-planar points to reduce the general PnP problem to a planar homography-based formulation, which simplifies computation while still retaining multiple valid solutions. Importantly for downstream processing, IPPE explicitly returns two candidate poses along with their reprojection errors, enabling post-hoc disambiguation or filtering based on additional information (e.g., temporal consistency, IMU data, or physical constraints). This makes it particularly suitable for sensor-fusion pipelines, such as combining AprilTag-based visual pose estimates with inertial measurements, where ambiguous visual solutions can be resolved using motion priors or dynamics.

D'Alfonso, L., Garone, E., Muraca, P., & Pugliese, P. (2021, May 14). *Camera and inertial sensor fusion for the PnP problem: algorithms and experimental results*. https://doi.org/10.1007/s00138-021-01219-0

Presents algorithms that combine visual and inertial data to solve the Perspective-n-Point (PnP) problem more robustly. Important for understanding how AprilTag pose estimation can be enhanced using IMU fusion to reduce ambiguity and drift in camera-only localization systems.

Dupree, M., Liu, X., & Zhu, Y. (2019). *RF Silent Drone Navigation* [Pamphlet]. UC Santa Barbara College of Engineering. https://web.ece.ucsb.edu/~yoga/capstone/static/img/projects/poster/visHawk_poster.pdf

Outlines a student project on drone navigation using optical tracking methods when GPS or RF communication is restricted. Demonstrates early-stage applications of visual fiducials (like AprilTags) for autonomous vehicle positioning without reliance on wireless signals.

Limelight Vision. (2024). *Robot Localization with MegaTag2*. Limelight Documentation. Retrieved September 19, 2025, from https://docs.limelightvision.io/docs/docs-limelight/pipeline-apriltag/apriltag-robot-localization-megatag2

Details Limelight's MegaTag2 pipeline for multi-tag localization in the FRC robotics context. Explains how multiple AprilTags improve pose accuracy and reduce ambiguity through IMU heading usage. A practical resource for real-time AprilTag-based localization pipelines in competitive robotics.

MATLAB & Simulink. (n.d.). *What Is SLAM? How it works, types of SLAM algorithms, and getting started*. MathWorks. Retrieved October 26, 2025, from https://www.mathworks.com/discovery/slam.html

This overview explains SLAM as a core computer vision and robotics technique used to estimate a robot's position while simultaneously building a map of its environment using sensor data such as cameras and LiDAR. It highlights that SLAM is widely used in real-time systems like autonomous robots and drones, where accurate and low-latency perception is required. This is relevant to improving AprilTag-based pose estimation with IMU fusion because both approaches address the same underlying problem in SLAM pipelines: combining visual measurements with additional sensors to improve robustness and reduce drift under motion, occlusion, or visual degradation.

Nugent, D. (2020, June 29). Apriltags 101. *Optitag*. https://optitag.io/blogs/news/apriltags-101

This overview explains AprilTags as fiducial markers designed for robust detection and 6-DOF pose estimation in camera-based systems. It emphasizes their use in robotics for real-time tracking due to their high detection reliability and efficient decoding, even under challenging imaging conditions. This is relevant to the paper's IMU–AprilTag fusion approach because it establishes AprilTags as a stable but vision-dependent pose reference, motivating sensor fusion to improve robustness under motion blur, occlusion, or intermittent visual loss.

Olson, E. (2011). *AprilTag: A robust and flexible visual fiducial system*. University of Michigan. https://docs.wpilib.org/en/stable/_downloads/50ad8f5c37aa2b4f174a3fde0fd71137/olson2011tags.pdf

The foundational research paper introducing the original AprilTag system. Describes tag design, detection pipeline, and pose estimation methods. Establishes the core mathematical and performance benchmarks that define fiducial-based localization today.

Olson, E., & Wang, J. (2016). *AprilTag 2: Efficient and robust fiducial detection*. Computer Science and Engineering Department at the University of Michigan in Ann Arbor, MI, USA.

https://doi.org/10.1109/iros.2016.7759617

Presents major algorithmic improvements to the AprilTag detection pipeline, including speed, robustness, and reduced false positives. Highly relevant as a primary reference for modern AprilTag

OpenCV. (n.d.). *Detection of ArUco Markers*. OpenCV - Open Source Computer Vision. Retrieved October 25, 2025, from

https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

Covers detection and pose estimation of ArUco markers, which are conceptually similar to AprilTags. Useful for comparing different fiducial marker systems and understanding shared mathematical foundations, such as homography and PnP pose estimation.

Pant, Y. V., Abbas, H., Mohta, K., Nghiem, T. X., Devietti, J., & Mangharam, R. (2016, January 14). *Co-design of Anytime Computation and Robust Control* (Proceedings - Real-Time Systems Symposium 1052-8725; Technical Report No. 9781467395076). Institute of Electrical and Electronics Engineers. https://doi.org/10.1109/rtss.2015.12

This paper evaluates how computational load from perception and estimation tasks in autonomous robotic systems can introduce latency into closed-loop control. The authors test their approach using a robotic control setup in which delayed state estimation affects

the robot's ability to respond accurately and maintain stability. The work is applicable to this paper because it demonstrates that computer vision and other computationally intensive processing pipelines must minimize latency and computational overhead in order to maintain fast and reliable real-time motion control.

Petersen, M. (2025, October 21). *Helios Vision System - Flame: Leveling the playing field* [Online forum post]. Chief Delphi. https://www.chiefdelphi.com/t/helios-vision-system-flame-leveling-the-playing-field/507349/10

Forum talks about an under-development computer vision system based around AprilTag detection for the FIRST robotics competition. Among the planned features listed by the author "Multi camera orchestration allowing several devices to work together in localization" stands out as an innovative way to expand AprilTag technology.

PhotonVision (Ed.). (2024). *AprilTag Pipeline Types*. PhotonVision Docs. Retrieved May 8, 2026, from https://docs.photonvision.org/en/v2025.3.1-rc1/docs/apriltag-pipelines/detector-types.html

Documentation for PhotonVision, a common software that integrates AprilTag localization for the FIRST robotics competition notes conflicting data stating both improved accuracy with the U-Michigan AprilTag detector (agrees with other source), while also stating "~2x slower than AruCo". In practice within PhotonVision, performance is roughly equal.

Raschka, S., Patterson, J., & Nolet, C. (n.d.). *Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence*.

https://doi.org/10.3390/info11040193

This survey paper reviews the dominant tools and practices in modern machine learning and data science, identifying Python as the most widely used programming language in these fields. It attributes this widespread adoption to Python's mature ecosystem of scientific and machine learning libraries (such as NumPy, SciPy, TensorFlow, PyTorch, and OpenCV), which significantly reduce development overhead and enable rapid prototyping. The paper is applicable to the claim about computer vision because it situates Python's ecosystem strength within real-world ML and CV workflows, showing that its library support and community adoption are key factors driving its prevalence in vision-based research and applications.

Rubloff, M. (2025, July 3). Postshot Adds April Tag Support. *Radiance Fields*. https://radiancefields.com/postshot-adds-april-tag-support

Reports on the integration of AprilTag detection into the Radiance Fields "Postshot" software. Highlights growing adoption of fiducial markers in 3D reconstruction and computer vision workflows, demonstrating cross-domain applicability beyond robotics.

Reports on the integration of AprilTag detection into the Radiance Fields "Postshot" software. Highlights growing adoption of fiducial markers in 3D reconstruction and computer vision workflows, demonstrating cross-domain applicability beyond robotics.

Rufus. (2019, December 19). *AprilTag vs Aruco markers* [Online forum post]. Stackexchange. https://robotics.stackexchange.com/questions/19901/apriltag-vs-aruco-markers

The author notes an anecdotal account of differences between the U-Michigan AprilTag and ArUco AprilTag detectors. The discussion highlights tradeoffs between the two systems, including computational performance, robustness, pose stability, and ease of

integration. Contributors note that the U-Mich AprilTag detector tends to perform better at long distances and exhibits less rotational ambiguity, while ArUco is often easier to integrate within OpenCV-based pipelines and may provide fewer false positives under some default configurations.

Scaramuzza, D. (2018). *Lecture 13 Visual Inertial Fusion* (Institute of Informatics). University of Zurich, ETH Zurich. https://rpg.ifi.uzh.ch/docs/teaching/2018/13_visual_inertial_fusion_advanced.pdf Academic lecture notes explaining how camera and IMU data are fused for accurate visual-inertial odometry. Relevant for understanding how AprilTag-based localization can be integrated into larger SLAM systems with inertial feedback to improve robustness.

Tan, S., Srinivasaragavan, A., Iturralde, K., & Holst, C. (2024). *Enhanced Precision in Built Environment Measurement: Integrating AprilTags Detection with Machine Learning*. Chair of Engineering Geodesy, School of Engineering and Design, Technical University of Munich, Chair of Digital Transformation in Construction, Institute of Construction Management, Faculty of Civil and Environmental Engineering, University of Stuttgart. https://www.iaarc.org/publications/fulltext/166_ISARC_2024_Paper_207.pdf Explores combining AprilTag detection with machine learning techniques to improve spatial measurement accuracy in construction and surveying. Demonstrates an applied, real-world use case of fiducial markers outside robotics, emphasizing measurement precision and automation.

WPILib. (2025, January 6). *What Are AprilTags?* FIRST Robotics Competition documentation. Retrieved October 26, 2025, from https://docs.wpilib.org/en/stable/docs/software/vision-processing/apriltag/apriltag-intro.ht

ml

Provides a robotics-focused overview of AprilTags for FRC teams. Explains how tags are used for robot localization, camera calibration, and coordinate estimation. A practical reference for real-world implementations of AprilTag vision systems in educational robotics.

Wu, P.-C., Tsai, Y.-H., & Chien, S.-Y. (2014, November). *Stable Pose Tracking from a Planar Target with an Analytical Motion Model in Real-time Applications*. Graduate Institute of Electronics Engineering National Taiwan University. https://docs.wpilib.org/en/stable/_downloads/3e68486d3b2afa7aaebd66ec49d0fb03/mmsp2014_spe.pdf

Introduces a mathematical framework for stable pose tracking using planar fiducials, addressing ambiguity and motion consistency. A foundational work for later AprilTag pose-stability research, including multi-frame tracking and analytical motion modeling.

Appendix A

Figure A1

Three detected AprilTags with decoded identifiers and 3-dimensional bounding box representing

3d estimate calculated by the PnP step. (Olson, 2011)

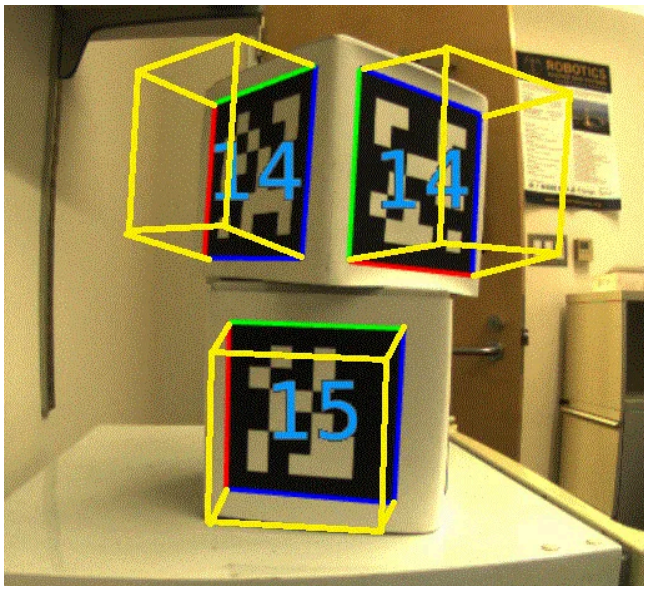


Figure A2

Two identical squares (simplification of the AprilTag pattern) with ambiguous 3d estimates. In both cases the calculated 3d estimate "looks correct" but in real-world conditions only one will be accurate.

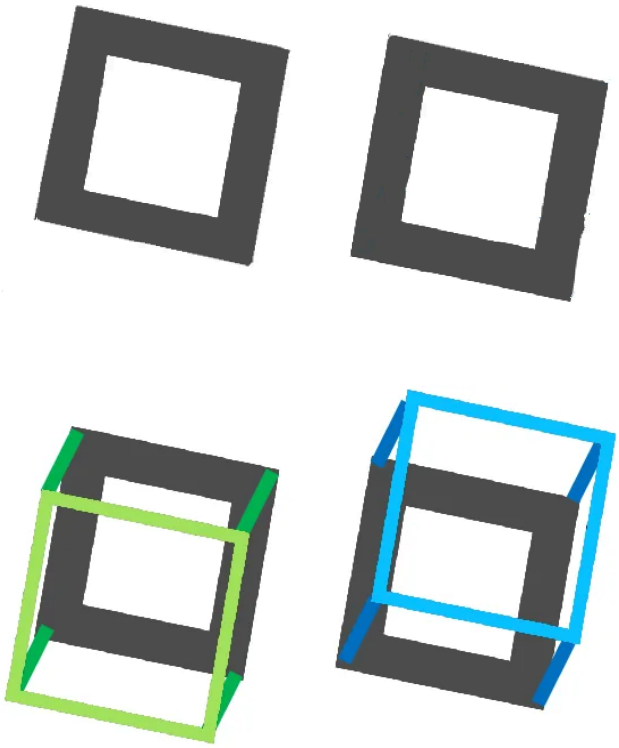


Figure A3

Diagram explaining each major step of the complete AprilTag pipeline. While it is not critical to understand the entire pipeline, the general pipeline is shown.

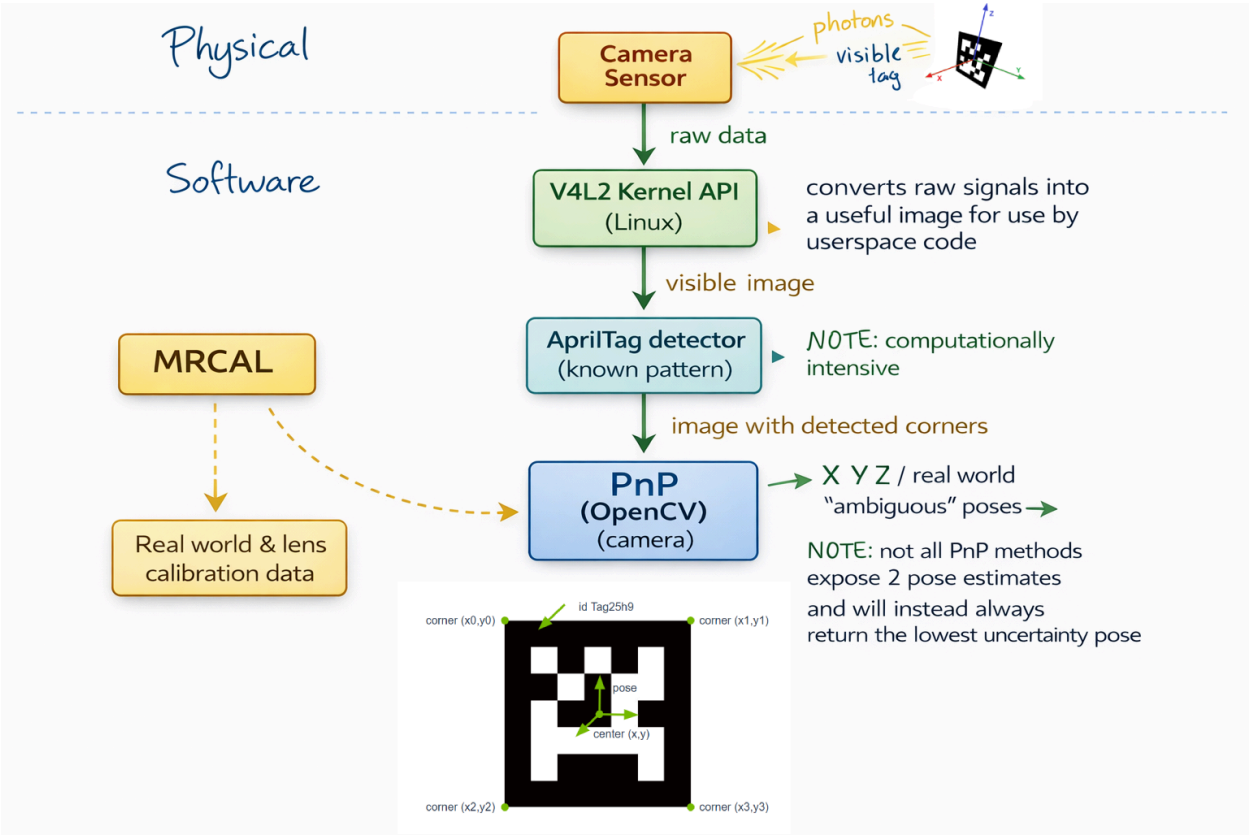


Figure A4

Experimental testing shows OpenCV apriltag and PnP running on a still image of an off-angle AprilTag. Both poses are shown with their error uncertainty and are available within code.

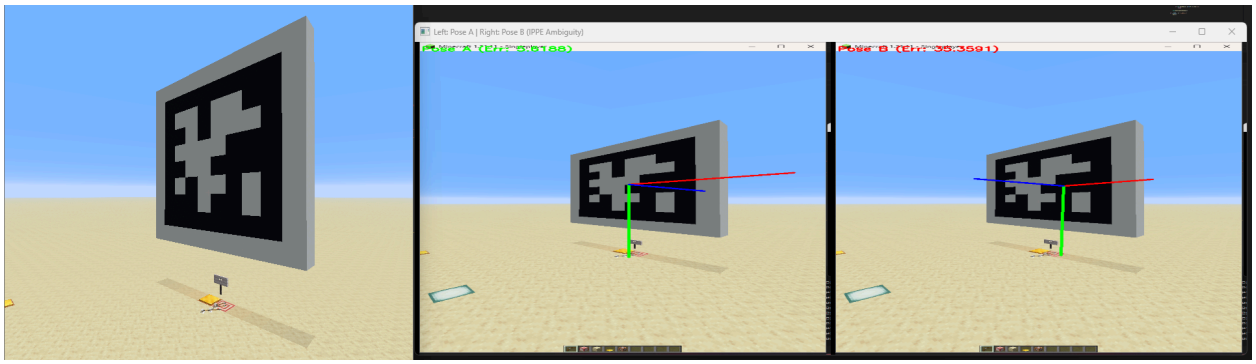
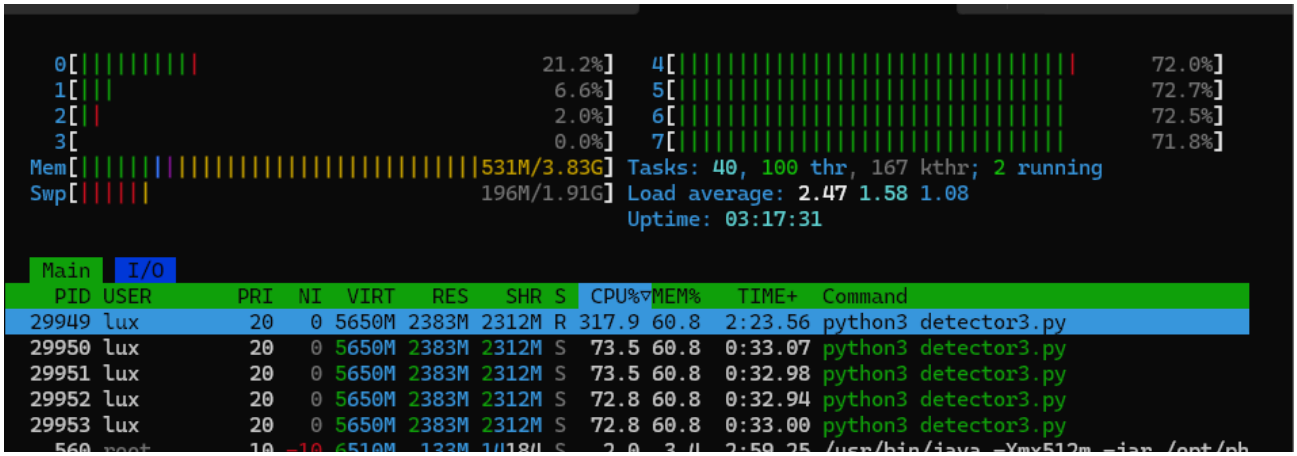


Figure A5

Linux htop is a tool commonly used to measure the processor utilization of different Linux applications, Shown is the baseline processor utilization of the U-michigan AprilTag detector alongside test cases running both 2 and 4 processes each with 4 threads allocated.


0[21.2%] 4[72.0%]
1[6.6%] 5[72.7%]
2[2.0%] 6[72.5%]
3[0.0%] 7[71.8%]
Mem[531M/3.83G] Tasks: 40, 100 thr, 167 kthr; 2 running
Swp[196M/1.91G] Load average: 2.47 1.58 1.08
Uptime: 03:17:31
Main I/O
PID USER PRI NI VIRT RES SHR S CPU%▽MEM% TIME+ Command
29949 lux 20 0 5650M 2383M 2312M R 317.9 60.8 2:23.56 python3 detector3.py
29950 lux 20 0 5650M 2383M 2312M S 73.5 60.8 0:33.07 python3 detector3.py
29951 lux 20 0 5650M 2383M 2312M S 73.5 60.8 0:32.98 python3 detector3.py
29952 lux 20 0 5650M 2383M 2312M S 72.8 60.8 0:32.94 python3 detector3.py
29953 lux 20 0 5650M 2383M 2312M S 72.8 60.8 0:33.00 python3 detector3.py

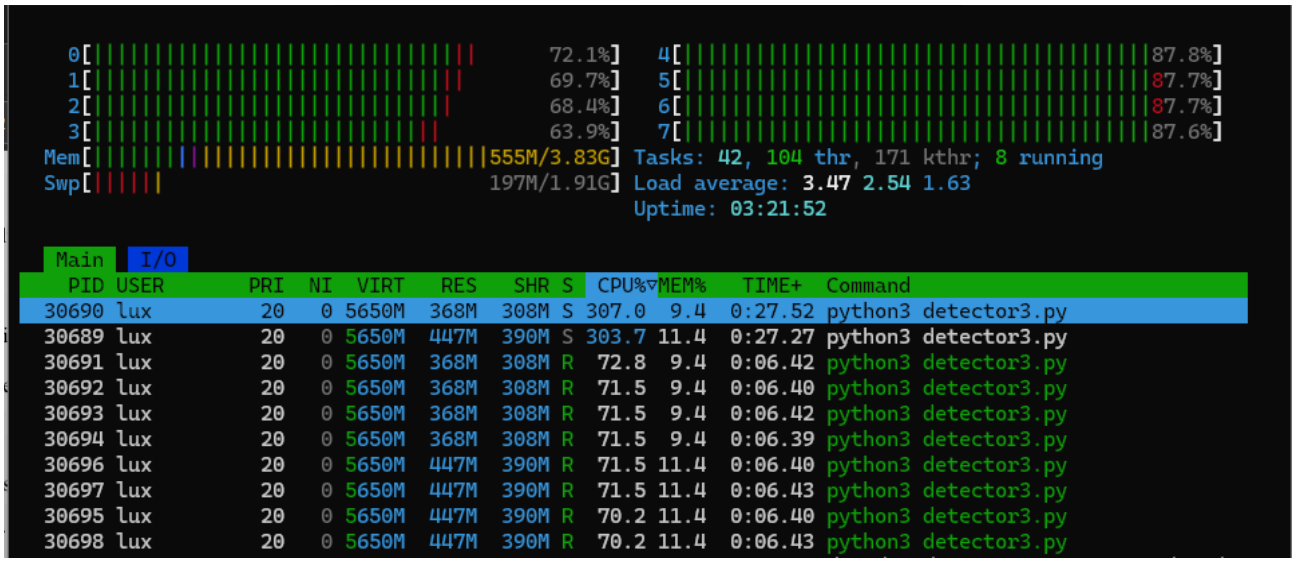

0[72.1%] 4[87.8%]
1[69.7%] 5[87.7%]
2[68.4%] 6[87.7%]
3[63.9%] 7[87.6%]
Mem[555M/3.83G] Tasks: 42, 104 thr, 171 kthr; 8 running
Swp[197M/1.91G] Load average: 3.47 2.54 1.63
Uptime: 03:21:52
Main I/O
PID USER PRI NI VIRT RES SHR S CPU%▽MEM% TIME+ Command
30690 lux 20 0 5650M 368M 308M S 307.0 9.4 0:27.52 python3 detector3.py
30689 lux 20 0 5650M 447M 390M S 303.7 11.4 0:27.27 python3 detector3.py
30691 lux 20 0 5650M 368M 308M R 72.8 9.4 0:06.42 python3 detector3.py
30692 lux 20 0 5650M 368M 308M R 71.5 9.4 0:06.40 python3 detector3.py
30693 lux 20 0 5650M 368M 308M R 71.5 9.4 0:06.42 python3 detector3.py
30694 lux 20 0 5650M 368M 308M R 71.5 9.4 0:06.39 python3 detector3.py
30696 lux 20 0 5650M 447M 390M R 71.5 11.4 0:06.40 python3 detector3.py
30697 lux 20 0 5650M 447M 390M R 71.5 11.4 0:06.43 python3 detector3.py
30695 lux 20 0 5650M 447M 390M R 70.2 11.4 0:06.40 python3 detector3.py
30698 lux 20 0 5650M 447M 390M R 70.2 11.4 0:06.43 python3 detector3.py

Figure A6

Plot demonstrating the scaling limitations of a singular instance of the AprilTag detector versus multiple instances of the detector (decimate 1 at 1920x1080)

Figure A7

Graph showing the relative performance uptick from updating the AprilTag detector from the latest full release (3.4.5) and the latest commit (3.4.5a) which includes several minor

optimizations to the code.

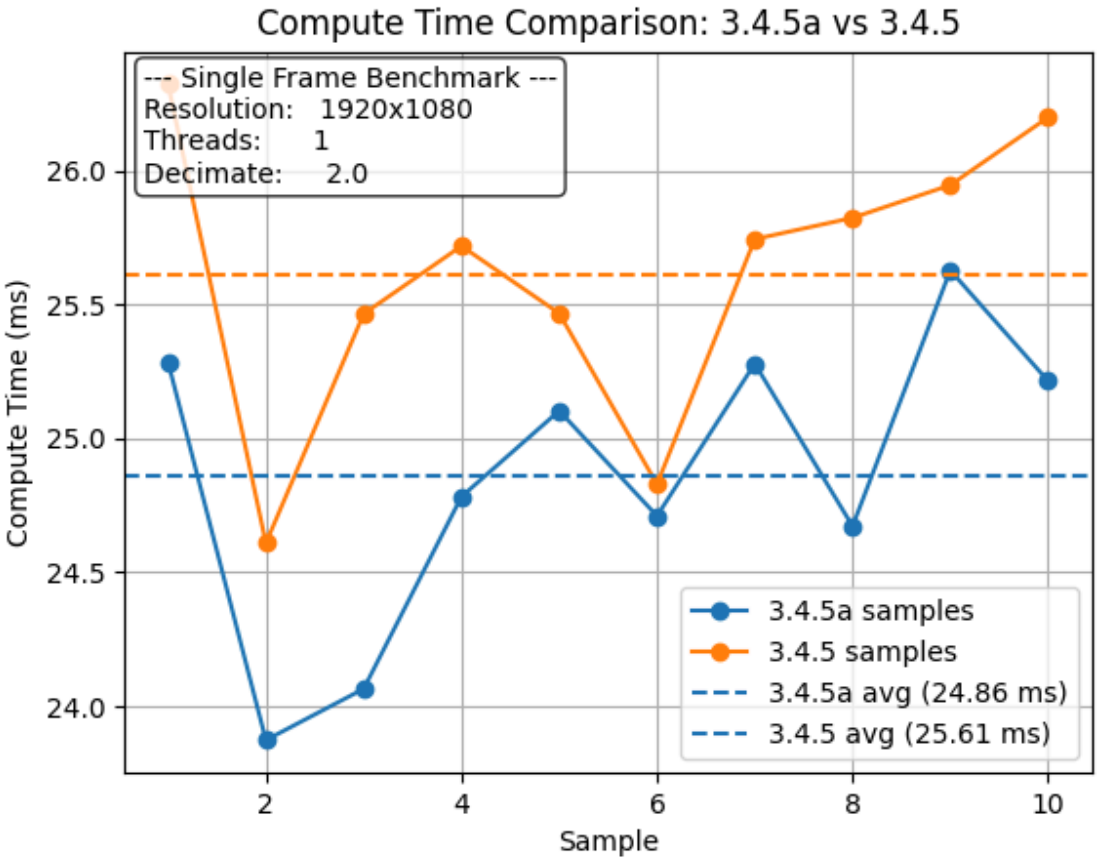
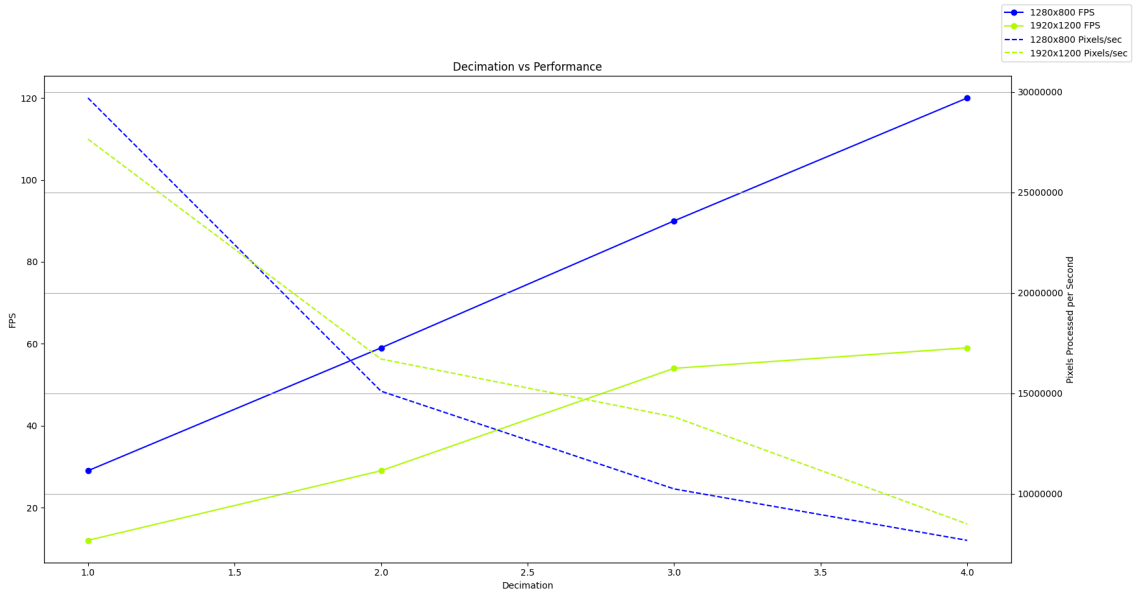
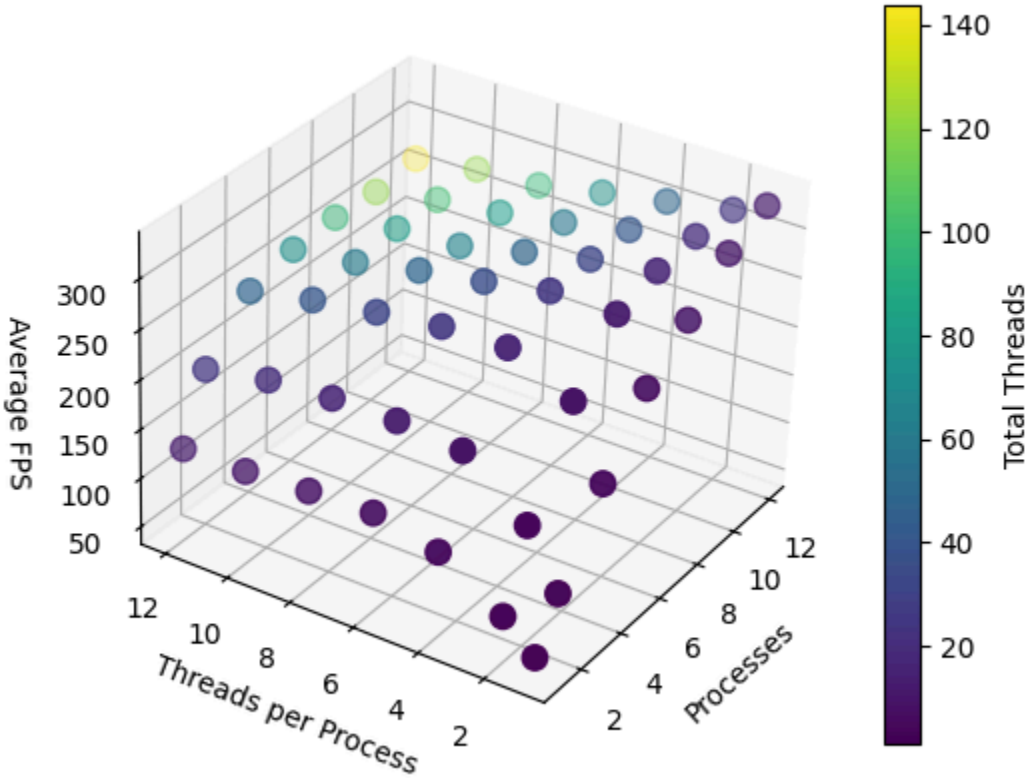


Figure A8

*AprilTag detector decimation vs performance at 1920x1200 and 860x600. Pixels processed per second has been extrapolated by taking res_x * res_y * fps for each point.*


1280x800 FPS
1920x1200 FPS
1280x800 Pixels/sec
1920x1200 Pixels/sec
Decimation vs Performance
FPS
Pixels Processed per Second
Decimation
120
100
80
60
40
20
30000000
25000000
20000000
15000000
10000000
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Average FPS
300
250
200
150
100
50
Threads per Process
12
10
8
6
4
2
Processes
12
10
8
6
4
2
Total Threads
140
120
100
80
60
40
20

Appendix B

1. My inquiry approach(es) are: **HIGHLIGHT** your choice(s)

Historical Research	Phenomenological	Narrative Research	Quasi-Experimental Research	Needs Assessment Research
Action Research	Case Study Research	Ethnographic	Descriptive	True Experimental
Content Analysis	Correlational Research	Evaluation Research	Grounded Theory	

2. My Data Collection Tool(s) are: **HIGHLIGHT** your choice(s)

Database	Field Notes	Interview	Document	Media
Focus Group	Measurement	Observation	Survey/Questionnaire	Text

How will you access your data: Google Docs, typed notes, computer screenshots (of results and proof), code repositories such as Github.

3. My Data Analysis Technique(s) are: **HIGHLIGHT** your choice(s)

Coding	Summarizing	Correlation & Regression	Descriptive Statistics	Inferential Statistics

4. The type of Data I'm collecting is: **HIGHLIGHT** your choice(s)

Quantitative Qualitative Mixed

5. The materials / equipment that I will need: **HIGHLIGHT** your choice(s)

Beakers	Computer
Microscopes	Measuring tape
Incubators	Camera
Videotape	Thermometer
Audio recordings	Other (list below):
Databases	Different integrated development environments (IDEs) and application-specific test software.(all of which are publicly available and free)
Consent Forms	

To determine IRB status...

7. Does your research involve human or animal subjects? In other words, are you conducting a focus group/observation/survey/questionnaire/interview? (highlight your choice) YES NO

(if you answer NO, you are exempt from the IRB and do not need to complete the section Part III: Research Application)

I. RESEARCH PROJECT IDENTIFICATION

A. Title of Proposed Research Project	Improving AprilTag Visual Localization Performance Using Visual and Inertial Data
B. Name of Primary Researcher	Anthony Andrews – Advanced Authentic Researcher, Electrical/ Computer-vision sub-team captain for Peninsula Robotics.
C. Collaborators (if applicable) **1. Teacher's name and e-mail address**	• Karen Logue – Advanced Authentic Research Teacher klogue@pausd.org • Jeffrey Fan – Technical Advisor (non-participant) Palo Alto High School Senior, Peninsula Robotics Software Captain

2. Consultant/Expert Adviser and email address	○ Jeffrey Fan will act only as an informal technical advisor. He will not collect data, be a subject, or be included as a participant in the study.
E. Purpose of Study	The problem this study investigates is technical, not human-based: ● Current AprilTag-based robot localization systems can produce: ● Ambiguous position estimates (multiple possible positions from the same image) ● Unstable outputs (jittering or jumping position values) ● Performance limitations (lower processing speed) ● These issues can reduce reliability and usability in robotics systems. **Purpose of the study:** ● To determine whether modifying existing AprilTag algorithms to include: ○ Inertial Measurement Unit (IMU) data ○ Multiple camera sources into the underlying Perspective'N'Point (PnP) algorithm. ● Can measurably: ○ Reduce positional ambiguity ○ Improve computational performance ○ Improve stability of localization results **Important clarification for IRB:** ● This study evaluates **software algorithms and robot sensor data only** ● It does **not** study people, behavior, opinions, or personal information
D. Context for Research	**D. CONTEXT FOR THE RESEARCH** ● AprilTags are visual markers (similar to QR codes) used by robots to determine position ● A camera detects the tag, and software estimates where the robot is located ● Under certain conditions, the math can return **multiple valid answers**, creating uncertainty ● Other robotics systems (such as SLAM) reduce this issue by combining: ○ Camera data ○ Motion sensor (IMU) data

	● This project applies similar ideas **only at the software level**, without human involvement ● Despite a high technical complexity I believe these changes are both feasible and realistically achievable in the given time because: ○ AprilTags and its underlying algorithms are public and open source ■ Its numerous algorithms, implementations, use of linear-algebra, trigonometry, low level computer programming, etc. ○ I have been researching and working with AprilTags as part of my involvement with a local community nonprofit FIRST robotics team (more details to follow) for 2 years. ■ General interest and experience with computer vision and CS. ○ I have a friend with additional technical expertise. ○ Unlike other localization technologies (GPS, SLAM) AprilTags are much more approachable and often used as a learning tool for the realm of computer vision and robotics.

II. RESEARCH GOALS

A. <u>Summary</u> Statement of Problem	RESEARCH GOALS Summary of the Problem: ● AprilTag localization can suffer from ambiguity and instability ● These issues are known but not well addressed in open-source implementations ● There is a gap in empirical testing of improved algorithms under controlled conditions
B. Research questions/hypotheses or specific objectives	B. Research Questions Can IMU data reduce ambiguity in AprilTag pose estimation? Can multi-camera data improve consistency and performance? How do modified algorithms compare to standard implementations?
C. Research Design	**C. RESEARCH DESIGN (STEP-BY-STEP, CLEAR)**

	Overall Design This is a **quasi-experimental, non-human, technical study**. The study follows this exact workflow: 1. **Identify baseline performance** ○ Use an existing, unmodified AprilTag software pipeline ○ Measure performance metrics 2. **Modify software algorithms** ○ Add IMU and/or multi-camera data to the computation ○ Change only one variable at a time 3. **Test under controlled conditions** ○ Simulated data (images or videos) ○ Real-world static robot tests 4. **Compare results** ○ Modified vs. unmodified software ○ Quantitative metrics only i. Qualitative analysis and conceptual analysis of results, room for future studies. 5. **Document and publish** ○ Results shared through a research paper and open-source code Github repository.

III. DETAILED DESCRIPTION OF PROCEDURES

A. Subjects needed and sampling procedure	**A. Subjects and Sampling Procedure**

	● **No human or animal subjects are involved** ● No surveys, interviews, observations, or behavioral data ● Peninsula Robotics lab will always have an adult supervisor whenever students are present. ● The "subjects" are: ○ Software algorithms ○ Robotic hardware components **IRB Clarification:** ● Human presence is incidental (supervision only) ● No human data is collected or analyzed ● Jeffrey Fan has agreed to give feedback on how well that localization works as the user of the localization data. He will provide feedback for performance, real-world integration, ease of use, and ideas. ● He has agreed to take on more of an advisor role and not be a part of the paper.
B. Approximate dates to begin and end data collection in/ through Palo Alto Unified School District	**Location:** Peninsula Robotics Lab Palo Alto Youth Robotics Association 2685 Middlefield Rd, Suite A, Palo Alto, CA 94306 Virtual/ entirely software based experimentation may also take place in other locations, however no physical hardware is utilized. **Dates:** Start: January 26, 2025 End: February 27, 2025
C. Amount of time required of participants	To address likely IRB safety and liability concerns: ● All testing occurs in a **private, enclosed robotics lab** ● **Adult mentors are present at all times** ● **Robots will NOT be moving** ○ Tests are static (robot powered but stationary) ○ Or fully simulated (software only)

	● No high-power mechanisms are operated
	● No modifications to safety systems
	● All participants in the space have already:
	  ○ Signed FIRST Robotics liability waivers
	  ○ Signed Palo Alto Youth Robotics Association waivers
	
	Additional safeguards:
	
	● No experimental hardware is shared with others
	● All custom hardware is:
	  ○ Owned by the researcher
	  ○ Low voltage
	  ○ Previously tested
	● No electrical, mechanical, or chemical hazards introduced

D. Instructions, instruments, or apparatus to be used (describe and attach copies)	Only **non-identifiable technical data** is collected: **Quantitative data:** ● Frames per second (FPS) ● Ambiguity ratio (how often multiple pose solutions appear) **Qualitative (technical notes only)** ● Software behavior observations ● Stability of algorithm output **Data sources:** ● Software logs ● Program outputs ● Test images and videos ● Robot sensor data (camera + IMU) **No personal data collected** ● No names ● No images of people ● No audio ● No identifying information
E. Technology to be used (infrastructure,	Hardware: ● E. Equipment and Technology Used ● Stationary robotics platform ● Cameras and inertial sensors

networking, hardware, software, etc)	● Laptop computer ● Open-source software: ● PhotonVision ● WPILib tools ● Custom analysis scripts ● Data logging via Google Sheets ● Code storage via GitHub All these items have been acquired already and
F. Specific activities and person(s) responsible for carrying out each activity	**Anthony Andrews** ● Conducts all testing ● Writes all code ● Collects and analyzes all data **Jeffrey Fan** ● Technical advisor only ● No data collection ● No subject participation

IV. RESULT OF RESEARCH

A. Rationale for the Study (How will the study contribute to this field of research?)	This study contributes by: ● Providing empirical data on AprilTag ambiguity reduction and performance gains. ● Evaluating algorithmic improvements under realistic conditions ● Supplying open-source implementations for public use to further the technology.
B. Benefits to the subjects	Because there are no traditional subjects, benefits instead apply to anyone looking to utilize the technology : ● Exposure to real research methods involving computer vision and algorithm evaluation ● Enhanced understanding of how their robot's vision system functions ● Opportunity to participate in meaningful STEM work ● Potential for improved robot performance in competitions No risks or personal data collection are involved.
C. Benefits to the community	**To the robotics community:** ● Improved localization reliability ● Better documented performance tradeoffs

	● Freely available software improvements ● As a key member of Peninsula Robotics, if the study is successful, the technology will be implemented to improve our team performance for the 2026 season. (Displaying tangible benefit to points scored and matches won) **To education:** ● Demonstrates ethical, low-risk technical research ● Encourages open-source STEM development
D. How will you use the information gained from the research?	● Academic research submission ● Open-source publication on GitHub ● Possible future extensions (outside this study)

Appendix C

Execution Plan for January 6 – February 27, 2025

Jan 6–9

- Time & Location: At home or at school
- People Involved: Myself
- Materials: Computer
- Tasks / Outcomes:
    - Research and document existing PnP strategies available in PhotonVision and related literature.

      Install and configure PhotonVision to run a stock AprilTag pipeline that will serve as the control implementation.

    - Verify that I can reproduce consistent baseline results (FPS, ambiguity behavior) under controlled conditions.
    - Read through key parts of the PhotonVision codebase to build a high-level understanding of the processing pipeline and where PnP is implemented.

Jan 12–16

- Time & Location: Home or school
- People Involved: Myself
- Materials: Computer
- Tasks / Outcomes:
    - Formulate several candidate PnP modifications or strategies based on theory, prior research, and baseline behavior.
    - Evaluate each proposed change for realism (doable within the project timeline) and potential impact (expected improvement over baseline).
    - Continue studying the PhotonVision codebase, using LLMs such as ChatGPT as a learning aid for understanding complex sections while still writing all experimental code myself.
    - Create a detailed diagram of the AprilTag–to–PnP pipeline for documentation and personal reference.

Jan 19–23

- Time & Location: Home or school

- People Involved: Myself
- Materials: Computer
- Tasks / Outcomes:
    - Begin implementing the first set of PnP modifications selected from the previous week.
    - Make incremental, well-documented code changes, testing after each step to ensure existing functionality is not broken.
    - Use test images as controlled sample data and vary only one factor (e.g., PnP strategy) at a time.
    - Record initial quantitative results (FPS, ambiguity ratio) and qualitative notes for each change.

Jan 26–30

- Time & Location: Home or school
- People Involved: Myself
- Materials: Computer
- Tasks / Outcomes
    - Continue experimentation with different methods, bounce ideas off of AI.
    - Extensively utilize the thorough and well documented libraries of OpenCV (which contains many proven and pre-written SolvePNP methods).

Feb 2–20

- Time & Location: Home or school
- People Involved: Myself,
- Materials: Computer
- Tasks / Outcomes:
    - Continue iterating on PnP modifications based on earlier results.
    - Systematically record quantitative data (FPS, ambiguity ratio) for each version in both simulated and real-world environments using Google Docs or Google Sheets.
    - Collect qualitative observations about behavior under challenging conditions such as:
        - Poor or uneven lighting
        - Partial occlusion or unstable visibility of AprilTags
        - Rapid motion of the robot and/or camera
    - If results are unsatisfactory, preserve existing code, reflect on failures, and design new modifications informed by the data and lessons learned.

Feb 23–27

- Time & Location: Home or school
- People Involved: Myself
- Materials: Computer
- Tasks / Outcomes:
    - Perform a final round of testing for both the stock (control) and best modified PnP implementations.

- Analyze the final dataset both quantitatively (FPS, ambiguity ratio, consistency across runs) and qualitatively (stability, reliability, usability).
- Explicitly compare the control and modified approaches, highlighting improvements, regressions, and context-dependent tradeoffs (e.g., performance under different lighting or motion).
- Reflect on key findings and identify possible directions for future work beyond the scope of this project.

Types of Data Collected

The study will primarily collect quantitative data, focusing on two main metrics produced by the AprilTag pipeline:

- Frames per second (FPS): A measure of computational throughput and real-time performance. (Additionally, for a single frame, time to process in milliseconds can be collected and used to extrapolate an FPS)
- Ambiguity ratio: A measure of how frequently or severely the system produces ambiguous pose estimates. For this project, the ambiguity ratio will be defined as the percentage of frames in which the AprilTag detector reports an ambiguous pose. Other PnP related observations will be anecdotal

In addition to these quantitative metrics, I will also collect qualitative data, including:

- Observations about ease of integration and configuration for different PnP strategies.

- Descriptions of specific failure cases (e.g., mis-detections in low light).

These qualitative notes will not be the primary focus of the analysis but will be used to interpret and contextualize the quantitative findings.